

SkyvoltRobot MVP

Phase 0 보고서 — 디지털 트윈 (Digital Twin)

작성일: 2026 - 06 - 15

1. Phase 0 개요

Phase 0는 SkyvoltRobot의 디지털 트윈 토대다. 두 갈래로 구성된다 — (가) ROS 없이 도는 순수 파이썬 알고리즘 코어(위치추정 칼만필터·도킹 속도정책 FSM·다중로봇 스케줄러)와 (나) Gazebo 3D 시뮬레이션(로봇·트랙·충전대 URDF + 주차장 월드). 이번 작업의 핵심은 이 토대를 '실제로 실행되게' 만들고 회귀 게이트로 검증한 것이다.

구성 요소	내용
알고리즘 코어 (IP)	순수 파이썬 — KF 위치추정, 속도정책 FSM, 플릿 스케줄러
평가 하니스	100회 도킹 시뮬레이션 + 회귀 게이트
Gazebo 디지털 트윈	로봇/트랙/충전대 URDF, parking_lot 월드
플릿 API 서버	FastAPI REST (작업 등록·상태 조회)

2. 알고리즘 코어 (순수 파이썬 IP)

- TrackLocalizer — 트랙 호장(arclength) 상태 [s, ds]의 칼만필터. 엔코더 오둠, RFID 태그, 포토아이 측정을 융합해 1D 위치를 추정한다.
- SpeedPolicyFSM — 2단 감속 도킹 FSM(CRUISE→APPROACH_1→APPROACH_2→DOCKED), 오버슈트 감시와 비상정지 포함.
- Scheduler + ReservationTable — 트랙 구간 시간창 예약 기반 다중로봇 그리디 스케줄러.

ROS 없이 도는 설계라 반복이 빠르고 CI에서 바로 게이트로 검증된다.

3. 단위 테스트 & 도킹 평가 하니스

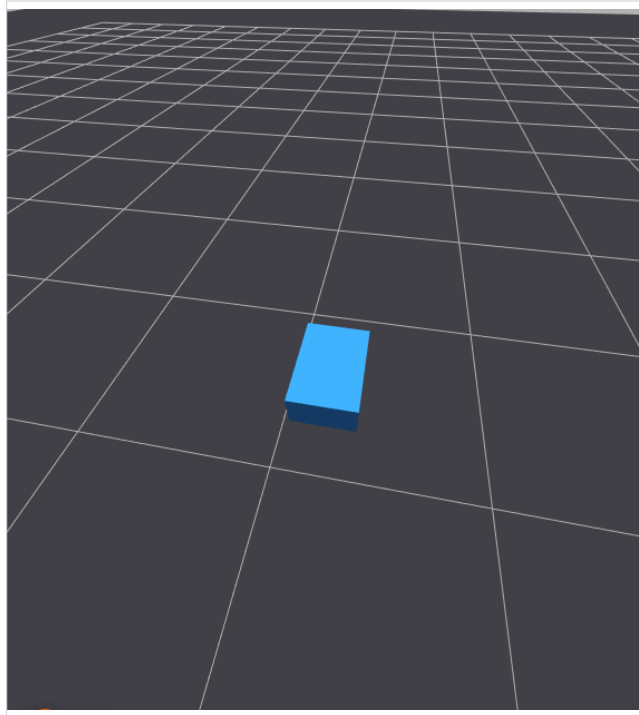
ROS의 launch_testing 플러그인이 최신 pytest와 충돌해 테스트 수집이 깨지는 문제를 발견, 서드파티 플러그인 자동로드를 비활성화하여 해결했다. Phase 0 코어 단위 테스트 22개가 통과하며(이후 단계에서 79개로 성장), 도킹 평가 하니스는 100회 시뮬레이션을 회귀 게이트로 검증한다.

지표 (100회 도킹)	측정값(p99)	합격 기준	판정
도킹 성공률	100 / 100	—	통과
횡방향 오차	약 2.9 mm	< 4 mm	통과
종방향 오차	약 9.2 mm	< 12 mm	통과
소요 시간	약 10.1 s	< 13 s	통과

최종 결과: PASS — 모든 게이트 충족.

4. Gazebo 디지털 트윈

ROS 2 Humble + Gazebo Fortress 환경을 설치하고 워크스페이스를 colcon으로 빌드했다. 주차장 월드(floor·ceiling·북/남 레일·지지대)에 SkyvoltRobot URDF를 스폰해 단일 로봇과 3대 플릿을 실시간(약 100%)으로 구동하는 데 성공했다.



Gazebo 주차장 월드에 스폰된 SkyvoltRobot (디지털 트윈)

5. 실행 과정에서 수정한 버그

#	증상	원인 / 조치
1	colcon 빌드 실패	없는 meshes 폴더 설치 시도 → CMakeLists에서 제거
2	노드 실행파일 누락	setup.cfg 부재로 bin/에 잘못 설치 → setup.cfg 추가
3	launch 파싱 실패	robot_description를 YAML로 오인 → ParameterValue(str) 래핑
4	로봇이 안 뜸	빈 링크에 질량 없어 모델 무시 → 관성 블록 추가(URDF)
5	3D 화면이 검은색	Ogre2가 NVIDIA에서 렌더 실패 → Ogre v1 + NVIDIA EGL 강제
6	테스트 수집 깨짐	ROS launch_testing 플러그인 충돌 → 자동로드 비활성화

이 중 #1~#4는 프로젝트 자체 버그, 나머지는 환경/도구 이슈였다.

6. 플릿 관리 API 서버

FastAPI 기반 REST 서버를 구동해(8080 포트) 충전 작업 등록 시 스케줄러가 로봇을 자동 배정하는 것을 확인했다. 서버 실행에 필요한 최소 파일을 자체 완결형 api-server/ 디렉토리로 모아 재사용성을 높였다. (Phase 2에서 읽기 전용 운영자 대시보드로 확장.)

7. 실행 방법

```
cd skyvolt_mvp
./scripts/run_tests.sh           # unit tests
./scripts/run_docking_eval.sh    # 100-trial docking eval -> PASS
./scripts/build_ros.sh           # build ROS workspace (first time)
./scripts/run_sim.sh [fleet]     # Gazebo single robot / 3-robot fleet
cd api-server && ./run.sh         # fleet API server (8080)
```

8. 결론

- 디지털 트윈 토대를 실제로 실행 가능하게 만들고 회귀 게이트로 검증했다 — 단위 테스트 통과, 도킹 평가 PASS, Gazebo 단일/플릿 구동, API 서버 동작.

- 여러 빌드·모델·렌더 버그를 수정해 재현 가능한 실행 환경(scripts/*.sh)을 갖췄다.
- 이 토대 위에서 Phase 1(위치추정 견고화)과 Phase 2(다중로봇 sim)를 진행했다.